

ARSH ADVANI

United States · 669-609-2927 · arshadvani3@gmail.com · [linkedin.com/in/arshadvani/](https://www.linkedin.com/in/arshadvani/) · github.com/arshadvani3

EDUCATION

University of California, Santa Cruz

Master of Science, Computer Science | GPA: 3.50/4.0

June 2026

Mukesh Patel School of Technology Management and Engineering, NMIMS

Bachelor of Technology, Computer Engineering

June 2024

SKILLS

Languages: Python, TypeScript, JavaScript, C++, SQL

AI / ML: LangGraph, LangChain, RAG, sentence-transformers, PyTorch, HuggingFace, Pydantic, Scikit-Learn

Backend & Web: FastAPI, Node.js, Express.js, Flask, Django, React, GraphQL, WebSockets, REST APIs

Cloud & DevOps: AWS (Lambda, S3, CloudFront), GCP, Docker, Kubernetes, GitHub Actions, CI/CD

Databases: PostgreSQL, Pinecone, MySQL, MongoDB, Redis, Qdrant, ChromaDB

WORK EXPERIENCE

BigShorts, Software Engineering Intern

June 2025 – September 2025

- Architected a content management backend in Node.js and MySQL serving 10,000+ users, supporting hierarchical series, seasons, and episodes with scheduled release logic across 6 JWT-authenticated REST APIs.
- Achieved sub-200ms response times across all endpoints through batch query optimization, platform-specific content delivery logic, and transaction management for concurrent write operations.
- Eliminated N+1 queries and applied strategic denormalization to improve database read performance by 40%, preserving integrity across 4 interconnected tables via application-level soft-delete cascades.
- Deployed a serverless AWS Lambda video pipeline converting 500+ uploads to HLS adaptive bitrate streams at 3 quality levels via FFmpeg and CloudFront CDN, cutting bandwidth consumption by 35%.

LTIMindtree, Data Science Intern

January 2024 – June 2024

- Automated enterprise diagram-to-text conversion using LLaVA vision-language models across 200+ workflow diagrams, cutting manual documentation effort by 40% and enabling downstream NLP pipelines on previously unstructured content.
- Benchmarked LLaMA, Mistral, and open-source LLMs across parameter efficiency, latency, and accuracy; reduced hallucination rates 30% through prompt engineering and informed internal LLM adoption strategy.

PROJECTS

AgentMesh - P2P Agent Discovery & Routing Framework | Python, FastAPI, LangGraph, WebSockets, React, TypeScript

- Architected a peer-to-peer agent discovery protocol where agents register capabilities at startup and are ranked at runtime via a 4-factor composite scoring formula (semantic similarity via sentence-transformers embeddings, EMA trust, availability, cost), replacing static orchestration with a self-tuning routing layer that shifts weights dynamically per caller signal (latency-sensitive callers boost availability weight, budget-sensitive callers boost cost weight) without pipeline rewrites.
- Built a capability-aware OutputEvaluator driving real EMA trust updates: deterministic grounding checks for structured outputs, LLM-as-judge for open-ended tasks, producing measurably differentiated agent reputation scores that compound across tasks and autonomously shift routing without configuration.
- Benchmarked across 5 controlled experiments: routing formula correctness (5/5), trust convergence, and resilience under agent failure, with 176 tests covering routing, trust scoring, circuit breaker state machine, auth, and MCP tool integration; measured 0.3–0.5s fixed mesh overhead over direct LLM latency.

AgentProbe - LLM Red-Teaming Platform | Python, FastAPI, LangGraph, React, TypeScript, PostgreSQL, Redis, Groq

- Developed a red-teaming platform that automatically stress-tests any LLM API for vulnerabilities and failure modes - architected as a 7-node LangGraph multi-agent system with conditional graph routing that auto-escalates attack difficulty when pass rate exceeds 90%, covering 5 failure dimensions.
- Implemented LLM-as-judge evaluation with the judge model strictly isolated from the system under test, running a 25-pattern injection battery across 7 attack categories and producing auditable per-dimension scores with narrative reports.

DevDocs AI - Production RAG System | Python, FastAPI, React, TypeScript, Qdrant, HuggingFace, Redis, Railway, Vercel

- Built a production documentation assistant that answers codebase questions from GitHub repos and uploaded files - AST-aware RAG pipeline across 27 file types, improving retrieval accuracy 40% over character-based splitting with per-chunk source citations.
- Cut repeat-query latency 95% via two-tier Upstash Redis cache sustaining 70% production hit rate; parallelized HuggingFace embedding calls 3x via `asyncio.gather` with semaphore control.